



**INSTITUTO POLITECNICO
NACIONAL**

ESIME CULHUACAN



Alumnos:

MUÑOZ LOZANO JATZIRY ALIN

SANCHEZ CRUZ ALEIDA ERANDI

MARTINEZ CLEMENTE DONOVAN ETIENNE

**BERNABE SOLARES KATTY ARIAFNA
(4EV24)**

SISTEMAS DE INFORMACION WEB

PROYECTO/ "S.W.A.T.D. R"

Profesor:

ENTZANA GARDUÑO YVENTZ

4EV14

S.W.A.T.D.R (Sistema Web de Administración para Tienda Departamental de Ropa)

-. Planteamiento del problema

Las tiendas de los grandes almacenes administran enormes cantidades de productos de marcas distintas, tallas, colores y categorías. En muchas ocasiones la gestión de existencias se realiza de forma manual o a través de sistemas poco interconectados, lo que produce errores en existencias, lentitud en las ventas y difíciles tareas para crear reportes, es necesario implementar un sistema de información web que permita gestionar con eficiencia el catálogo de prendas, el inventario, los proveedores, los clientes y las ventas que se han producido, facilitando así la consulta y la actualización de la información en tiempo real.

El sistema permite disminuir errores administrativos, obtener un mayor control de existencias y una rapidez en el proceso de venta, y nuestro objetivo principal es Desarrollar un sistema web para la administración de una tienda departamental de ropa que permita gestionar productos, inventarios, proveedores, clientes y ventas mediante una base de datos centralizada.

-. PostgreSQL 16 & pgAdmin4 POSTRESS (BASE DE DATOS)

La base de datos denominada SWATDR (Sistema Web de Administración para Tienda Departamental de Ropa) fue implementada en PostgreSQL, también conocido como RDBMS (Relational Database Management System). Por su estructura y funcionamiento se considera una base de datos relacional del tipo OLTP (Online Transaction Processing) ya que está destinada a registrar las operaciones que se producen en tiempo real a partir de una aplicación web como pueden ser el registro de usuarios, el log-in en el sistema, la consulta de productos, la gestión del inventario o los pedidos. La información se encuentra almacenada en tablas relacionadas a partir de las relaciones de tipo: llave primaria / llave foránea la cual permite garantizar la integridad de los datos de la base de datos y, al mismo tiempo, evitar la duplicidad de la información, al iniciar el script, lo primero que se realiza es la activación de la extensión **pgcrypto** mediante la instrucción **CREATE EXTENSION IF NOT EXISTS pgcrypto;**. Esta extensión añade funciones criptográficas a PostgreSQL, siendo utilizada principalmente para proteger las contraseñas de los usuarios. En lugar de

almacenar las contraseñas en texto plano, el sistema las cifra utilizando el algoritmo bcrypt por medio de las funciones `crypt()` y `gen_salt('bf')`. Gracias a esto, aunque una persona obtuviera acceso a la base de datos, no podría conocer las contraseñas originales, ya que únicamente se almacenan sus representaciones cifradas. Esta práctica constituye uno de los mecanismos de seguridad más utilizados actualmente en aplicaciones web.

Posteriormente, el script realiza una limpieza completa de todos los objetos existentes relacionados con el proyecto. Se eliminan vistas, funciones, disparadores (triggers) y tablas mediante instrucciones `DROP ... IF EXISTS`, utilizando además la cláusula `CASCADE` para eliminar automáticamente cualquier dependencia. Esta etapa tiene como finalidad permitir que el script pueda ejecutarse repetidas veces durante el desarrollo sin generar errores por la existencia previa de objetos con el mismo nombre. Es una práctica común cuando se desea reconstruir completamente la base de datos desde cero.

Una vez preparada la base de datos, comienza la creación de las tablas que conforman el modelo relacional del sistema. La primera tabla creada es **roles**, cuya función consiste en almacenar los distintos tipos de usuarios que pueden existir dentro del sistema, en este caso "Administrador" y "Comprador". En lugar de guardar el nombre del rol en cada usuario, únicamente se almacena el identificador del rol, lo que reduce la redundancia de datos y cumple con los principios de normalización de bases de datos.

Después se crea la tabla **usuarios**, considerada el núcleo del sistema de autenticación. En ella se almacena el nombre del usuario, su correo electrónico, la contraseña cifrada, el rol asignado, la fecha de registro y dos campos adicionales relacionados con la seguridad: el número de intentos fallidos de acceso y la fecha hasta la cual la cuenta permanecerá bloqueada en caso de exceder el límite permitido. La relación entre usuarios y roles se establece mediante una llave foránea, garantizando que únicamente puedan asignarse roles válidos registrados previamente.

La siguiente tabla corresponde a **categorías**, cuya finalidad es clasificar los productos comercializados por la tienda. En este proyecto existen categorías como Caballeros, Damas y Accesorios. Esta separación permite organizar de forma eficiente el catálogo y facilita las búsquedas posteriores. Además, evita repetir el nombre de la categoría en cada producto, contribuyendo nuevamente a la normalización de la base de datos.

Posteriormente se crea la tabla **productos**, donde se almacena la información general de cada artículo disponible en la tienda. Cada registro contiene el nombre del producto, una descripción, la categoría a la que pertenece, su precio de mercado, la marca y la fecha en la que fue dado de alta en el sistema. Cada producto mantiene una relación directa con la

tabla categorías mediante una llave foránea, estableciendo así una relación de uno a muchos: una categoría puede contener numerosos productos, pero cada producto pertenece únicamente a una categoría.

Sin embargo, un mismo producto puede comercializarse en diferentes tallas y colores. Para resolver esta situación se implementa la tabla **variantes_producto**, la cual representa una especialización de los productos. Cada variante almacena la talla, el color y el stock disponible de una combinación específica. Por ejemplo, una chamarra puede existir en talla mediana y color azul, así como en talla grande y color negro, siendo cada combinación un registro independiente. Además, el sistema impide que existan dos variantes exactamente iguales mediante una restricción UNIQUE que combina producto, talla y color. Esta estructura resulta mucho más flexible que almacenar todos los atributos en una sola tabla. Posteriormente se crean las tablas **pedidos y detalles_pedido**, las cuales representan las ventas realizadas por los clientes. La tabla pedidos almacena únicamente la información general de la compra, como el comprador, la fecha y el monto total. Por otra parte, la tabla detalles_pedido registra cada uno de los productos adquiridos dentro de ese pedido, indicando la variante específica, la cantidad comprada y el precio unitario. Esta separación responde al modelo clásico de cabecera-detalle utilizado en prácticamente todos los sistemas comerciales y permite que un mismo pedido contenga múltiples artículos diferentes.

Después de construir el modelo principal, el sistema incorpora una tabla denominada **logs_sesion**, destinada exclusivamente a la auditoría de accesos. Cada vez que un usuario intenta iniciar sesión, independientemente de que lo consiga o no, queda registrado un evento que almacena el usuario involucrado, el correo electrónico utilizado, la fecha y hora del intento, la dirección IP, el navegador empleado, el estado del acceso (exitoso o fallido) y un mensaje descriptivo. Esta información resulta fundamental para fines de seguridad, ya que permite detectar intentos de acceso indebidos, monitorear la actividad de los usuarios y mantener un historial completo de autenticación.

El siguiente componente importante corresponde a la implementación de un **trigger**, denominado trg_mensajeria_stock. Un trigger es un mecanismo que permite ejecutar automáticamente una acción cuando ocurre un determinado evento sobre una tabla. En este caso, cada vez que se inserta o actualiza una variante de producto, el trigger verifica el nivel de existencias. Si el stock llega a cero, el sistema genera una alerta indicando que el producto se encuentra agotado; si el stock es menor a diez unidades, emite una advertencia recomendando su reabastecimiento. Gracias a este mecanismo no es

necesario que un administrador revise manualmente el inventario constantemente, pues la base de datos detecta automáticamente estas situaciones.

Posteriormente se crean varios índices sobre las columnas más utilizadas durante las consultas, como el nombre del producto, la marca, la categoría y el historial de sesiones. Los índices funcionan de manera similar al índice de un libro: en lugar de recorrer toda la tabla buscando un dato específico, PostgreSQL puede localizarlo mucho más rápidamente, reduciendo considerablemente el tiempo de respuesta de las consultas cuando el volumen de información aumenta.

Una vez definida toda la estructura, el script inserta datos de prueba. Se crean inicialmente los roles de Administrador y Comprador, posteriormente algunos usuarios con sus contraseñas cifradas, diversas categorías de ropa, varios productos pertenecientes a diferentes marcas, sus respectivas variantes de talla y color, un pedido de ejemplo y el detalle correspondiente a dicha compra. Estos registros permiten comprobar inmediatamente el funcionamiento del sistema sin necesidad de capturar información manualmente.

El bloque siguiente implementa las funciones principales que utilizará la aplicación web. La función **buscar_ropa_web** permite realizar búsquedas dinámicas filtrando por nombre del producto, categoría o precio máximo. Internamente une las tablas productos, categorías y variantes, calcula el stock total disponible y concatena las tallas existentes para presentar una respuesta completa al usuario. Además, únicamente devuelve aquellos productos cuyo inventario total sea mayor que cero, evitando mostrar artículos completamente agotados. La función **autenticar_usuario** constituye el núcleo del sistema de inicio de sesión. Cuando un usuario intenta acceder, primero verifica que el correo exista dentro de la base de datos. Si no existe, registra inmediatamente el intento fallido en la tabla de auditoría. Si el usuario existe, comprueba si la cuenta se encuentra temporalmente bloqueada. En caso contrario, compara la contraseña introducida con la contraseña cifrada almacenada utilizando nuevamente la función `crypt()`. Si ambas coinciden, registra un acceso exitoso, reinicia el contador de intentos fallidos y permite el ingreso. Si la contraseña es incorrecta, incrementa el número de intentos fallidos y, al llegar a cinco errores consecutivos, bloquea automáticamente la cuenta durante treinta minutos. Todo este proceso ocurre completamente dentro de PostgreSQL, sin depender de la aplicación web, aumentando considerablemente la seguridad del sistema.

La función **registrar_usuario** automatiza el proceso de creación de nuevos usuarios. Antes de insertar la información verifica que el correo electrónico no exista previamente y que el

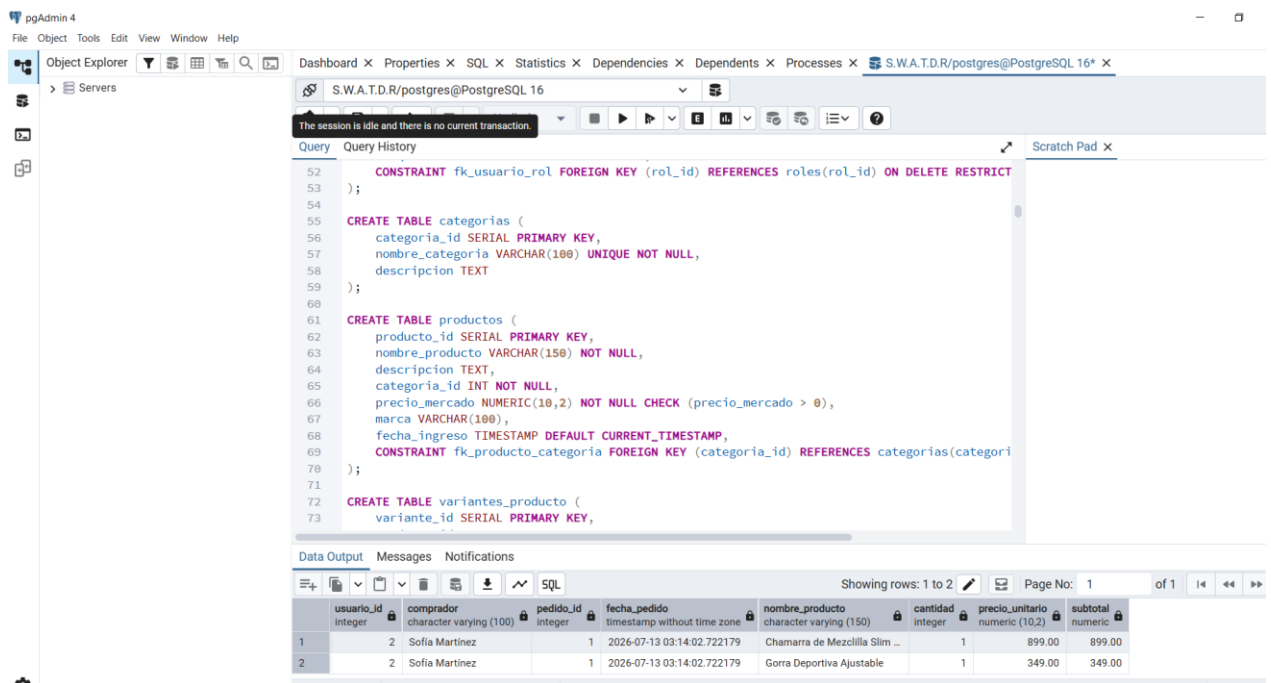
rol solicitado sea válido. Posteriormente cifra la contraseña mediante bcrypt, crea el nuevo registro en la tabla usuarios y finalmente genera un registro dentro del historial de auditoría indicando que el usuario fue registrado exitosamente.

Por otra parte, la función **obtener_historial_logins** permite consultar los accesos realizados por un usuario específico, mostrando la fecha del intento, el resultado obtenido, la dirección IP desde la cual se realizó la conexión y un mensaje descriptivo. Esta función facilita el monitoreo del comportamiento de cada usuario y complementa el sistema de auditoría implementado.

Finalmente, el script crea tres vistas destinadas a simplificar consultas frecuentes. La primera, **vista_panel_administrador**, proporciona una visión completa del inventario indicando el estado de cada variante como "Stock correcto", "Reabastecer urgente" o "Agotado", facilitando la administración del almacén. La segunda,

vista_historial_comprador, reúne en una sola consulta toda la información relacionada con las compras realizadas por cada cliente, mostrando pedidos, productos adquiridos, cantidades y subtotales. La tercera, **vista_auditoria_logins**, presenta un historial organizado de todos los intentos de acceso registrados por el sistema, incluyendo información del usuario, dirección IP, navegador, estado del inicio de sesión y mensaje correspondiente.

En conjunto, la base de datos SWATDR presenta una estructura sólida y correctamente normalizada, aplicando mecanismos modernos de seguridad mediante cifrado de contraseñas, control de intentos fallidos, bloqueo automático de cuentas, auditoría de accesos, automatización mediante triggers y optimización del rendimiento mediante índices. Estas características la convierten en una base de datos adecuada para soportar una aplicación web de comercio electrónico, garantizando integridad, seguridad, eficiencia y facilidad de mantenimiento.



(SCRIPT SQL)

- Servidor Backend (server.js)

El archivo server.js constituye el backend del sistema SWATDR. Fue desarrollado utilizando Node.js como entorno de ejecución y Express.js como framework para el desarrollo de aplicaciones web. Todo el código fue implementado en Visual Studio Code, un entorno de desarrollo integrado (IDE) ampliamente utilizado para proyectos basados en JavaScript debido a sus herramientas de depuración, administración de extensiones e integración con Node.js.

Desde el punto de vista de la arquitectura de software, este backend forma parte de una arquitectura Cliente-Servidor de tres capas (Three-Tier Architecture). La primera capa corresponde al cliente o frontend, donde el usuario interactúa con la aplicación mediante un navegador web; la segunda capa es el backend, implementado en el archivo server.js, encargado de procesar las solicitudes HTTP, validar la información y coordinar la lógica del sistema; finalmente, la tercera capa corresponde a la base de datos PostgreSQL, donde se almacena toda la información del sistema. De esta forma, el backend actúa como intermediario entre la interfaz gráfica y la base de datos, evitando que el cliente tenga acceso directo a la información almacenada.

El proyecto utiliza el entorno de ejecución Node.js, el cual permite ejecutar código JavaScript fuera del navegador. Gracias a ello, el mismo lenguaje puede utilizarse tanto en el desarrollo del frontend como del backend, facilitando el mantenimiento del sistema y reduciendo la complejidad del desarrollo. Sobre Node.js se implementa Express.js, un

framework diseñado para simplificar la creación de servidores web y servicios REST, proporcionando herramientas para definir rutas, recibir solicitudes, enviar respuestas y administrar diferentes tipos de contenido.

Al comienzo del archivo se importan todas las dependencias necesarias mediante la función `require()`. Se carga la biblioteca Express, encargada de crear el servidor web; Cors, que permite controlar el intercambio de información entre aplicaciones ubicadas en diferentes dominios; el archivo `db.js`, que contiene la configuración de la conexión con PostgreSQL mediante un pool de conexiones; además de las bibliotecas `Swagger-jsdoc` y `Swagger-ui-express`, utilizadas para generar automáticamente la documentación de la API. Posteriormente se crea la aplicación principal mediante la instrucción `const app = express();`, la cual representa el servidor que recibirá todas las solicitudes enviadas por los clientes. También se define el puerto de comunicación utilizando la constante `PORT = 3000`, indicando que el servidor escuchará las peticiones en el puerto 3000 del equipo donde se encuentre ejecutándose.

Uno de los primeros componentes configurados es Swagger, herramienta que permite documentar automáticamente todos los servicios web disponibles en la aplicación. Mediante un objeto de configuración se especifica la versión del estándar OpenAPI, el nombre de la aplicación, su versión, una descripción general y la dirección del servidor. Posteriormente, Swagger genera una interfaz gráfica accesible desde la ruta `/api-docs`, donde es posible consultar todos los servicios disponibles, sus parámetros de entrada y las respuestas que devuelve cada uno. Esto facilita considerablemente las pruebas y el mantenimiento del sistema, ya que cualquier desarrollador puede conocer el funcionamiento de la API sin necesidad de revisar directamente el código fuente.

Después de configurar Swagger, el servidor incorpora varios middlewares, los cuales son funciones que procesan las solicitudes antes de que lleguen a las rutas principales. El middleware `express.static('public')` permite publicar archivos estáticos como páginas HTML, hojas de estilo CSS, imágenes y archivos JavaScript del frontend. El middleware `cors()` habilita el intercambio de información entre distintos dominios, evitando restricciones de seguridad impuestas por los navegadores cuando el frontend y el backend se ejecutan en servidores diferentes. Finalmente, `express.json()` permite interpretar automáticamente la información enviada por el cliente en formato JSON, convirtiéndola en objetos JavaScript para facilitar su procesamiento.

La primera ruta implementada corresponde a la ruta principal (`/`). Cuando un usuario accede

al servidor desde su navegador, Express responde enviando el archivo `index.html` ubicado dentro de la carpeta `public`, convirtiéndose en el punto de entrada de toda la aplicación web.

A continuación se implementa el módulo de autenticación. La ruta `/api/auth/login` recibe mediante el método HTTP POST el correo electrónico, la contraseña, la dirección IP y el navegador utilizado por el cliente. Estos datos son enviados directamente a la función almacenada `autenticar_usuario()` creada previamente en PostgreSQL. El backend no valida las credenciales por sí mismo, sino que delega completamente esta responsabilidad a la base de datos. Una vez obtenida la respuesta, el servidor analiza el resultado y devuelve al cliente un objeto JSON indicando si el acceso fue exitoso, junto con la información del usuario o el mensaje correspondiente en caso de error. Esta estrategia centraliza toda la lógica de autenticación en un único lugar y reduce la posibilidad de inconsistencias entre la aplicación y la base de datos.

El proceso de registro funciona de manera similar mediante la ruta `/api/auth/register`. Cuando un nuevo usuario envía su información, el backend invoca la función `registrar_usuario()` almacenada en PostgreSQL. La base de datos verifica que el correo no exista previamente, cifra la contraseña utilizando `bcrypt`, crea el nuevo registro y devuelve un mensaje indicando si el proceso fue exitoso. El servidor únicamente recibe dicha respuesta y la transforma en un objeto JSON que será enviado nuevamente al cliente.

El backend también incorpora diversos servicios relacionados con la auditoría y el monitoreo del sistema. La ruta `/api/auth/auditoria` consulta la vista `vista_auditoria_logins`, mostrando todos los intentos de acceso registrados. De manera similar, la ruta `/api/auth/historial/:usuario_id` obtiene el historial de inicios de sesión correspondiente a un usuario específico mediante la función `obtener_historial_logins()`. Estas rutas permiten supervisar el comportamiento de los usuarios y detectar posibles accesos no autorizados.

En cuanto al catálogo de productos, el backend implementa varias rutas destinadas a consultar la información almacenada en la base de datos. La ruta `/api/productos` obtiene el catálogo completo utilizando la función `buscar_ropa_web()`, mientras que `/api/buscar` permite realizar búsquedas filtrando por nombre, categoría o precio máximo. En ambos casos, Express recibe los parámetros enviados por el cliente, ejecuta la consulta correspondiente sobre PostgreSQL y devuelve los resultados en formato JSON para que posteriormente sean mostrados en la interfaz web.

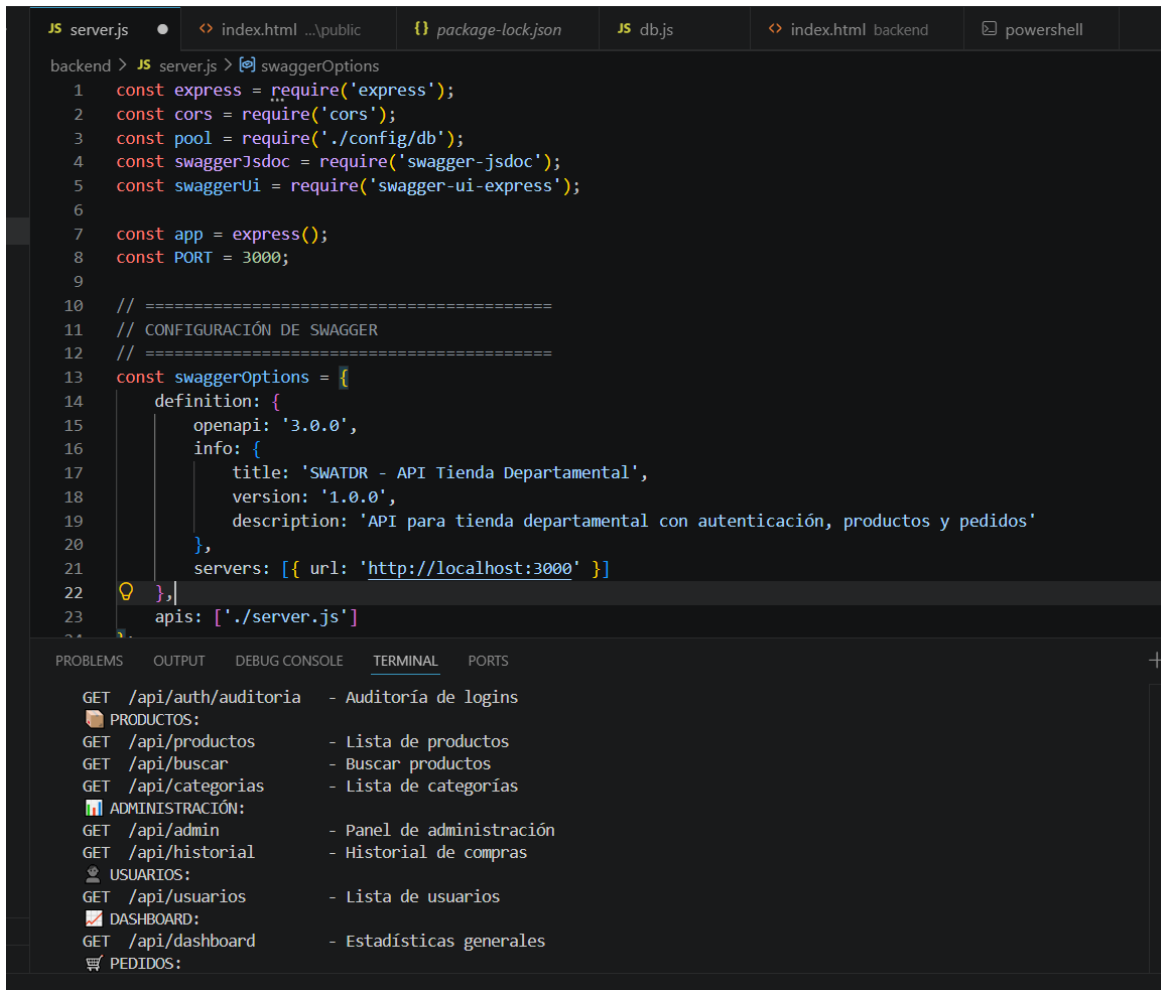
Además del catálogo, el sistema dispone de rutas específicas para consultar categorías, usuarios registrados, panel administrativo e historial de compras. Cada una de estas rutas

ejecuta consultas SQL o utiliza vistas previamente creadas dentro de la base de datos, reduciendo la complejidad del código y aprovechando la lógica implementada directamente en PostgreSQL.

Otro módulo importante corresponde al Dashboard del sistema. La ruta `/api/dashboard` ejecuta varias consultas independientes para contar el número total de productos, usuarios y pedidos registrados, así como calcular el monto total de ingresos obtenidos por la tienda. Posteriormente reúne toda esta información en un único objeto JSON que puede utilizarse para alimentar paneles estadísticos dentro del sistema administrativo.

La creación de pedidos constituye una de las operaciones más relevantes del backend. Cuando un cliente confirma una compra, la ruta `/api/pedidos` recibe la información correspondiente al comprador, el importe total y el listado de productos adquiridos. Primero se registra el pedido principal dentro de la tabla `pedidos`, obteniendo automáticamente el identificador generado por PostgreSQL. Después se recorren todos los productos enviados por el cliente mediante un ciclo `for`, insertando cada uno dentro de la tabla `detalles_pedido`. De esta manera se conserva correctamente la relación entre el pedido y todos los artículos adquiridos, reproduciendo el modelo clásico de cabecera y detalle utilizado en sistemas comerciales.

Finalmente, el servidor se inicia mediante la instrucción `app.listen(PORT)`, la cual pone en funcionamiento el servicio web y comienza a escuchar solicitudes en el puerto 3000. Durante el arranque se muestran en la consola diferentes mensajes informativos indicando la dirección del servidor, el acceso a la documentación generada por Swagger y la lista completa de los endpoints disponibles, facilitando las pruebas y el desarrollo del sistema. En conjunto, el archivo `server.js` constituye el núcleo del backend del proyecto SWATDR. Su principal responsabilidad consiste en recibir las solicitudes del cliente, procesarlas mediante Express, comunicarse con la base de datos PostgreSQL utilizando consultas SQL y funciones almacenadas, y devolver respuestas estructuradas en formato JSON. La implementación utiliza una arquitectura cliente-servidor moderna basada en servicios REST, integra documentación automática mediante Swagger y delega gran parte de la lógica de negocio a la base de datos, logrando un sistema organizado, seguro, escalable y fácil de mantener.



```
backend > JS server.js > swaggerOptions
1  const express = require('express');
2  const cors = require('cors');
3  const pool = require('./config/db');
4  const swaggerJsdoc = require('swagger-jsdoc');
5  const swaggerUi = require('swagger-ui-express');
6
7  const app = express();
8  const PORT = 3000;
9
10 // =====
11 // CONFIGURACIÓN DE SWAGGER
12 // =====
13 const swaggerOptions = {
14   definition: {
15     openapi: '3.0.0',
16     info: {
17       title: 'SWATDR - API Tienda Departamental',
18       version: '1.0.0',
19       description: 'API para tienda departamental con autenticación, productos y pedidos'
20     },
21     servers: [{ url: 'http://localhost:3000' }]
22   },
23   apis: ['./server.js']
24 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- GET /api/auth/auditoria - Auditoría de logins
- PRODUCTOS:
- GET /api/productos - Lista de productos
- GET /api/buscar - Buscar productos
- GET /api/categorias - Lista de categorías
- ADMINISTRACIÓN:
- GET /api/admin - Panel de administración
- GET /api/historial - Historial de compras
- USUARIOS:
- GET /api/usuarios - Lista de usuarios
- DASHBOARD:
- GET /api/dashboard - Estadísticas generales
- PEDIDOS:

(server. Js en visual code)

- Configuración de la Conexión a la Base de Datos (db.js)

El archivo db.js constituye el módulo encargado de establecer la comunicación entre el backend desarrollado en Node.js y la base de datos PostgreSQL. Su principal función es crear una conexión segura y reutilizable que permita al servidor ejecutar consultas SQL, llamar funciones almacenadas y acceder a toda la información del sistema SWATDR. Este archivo representa el punto de enlace entre la capa lógica de la aplicación y la capa de almacenamiento de datos, siendo indispensable para el funcionamiento de todos los servicios implementados en el backend.

Al inicio del archivo se importa la biblioteca pg mediante la instrucción `const { Pool } = require('pg');`. Esta biblioteca es el controlador oficial de PostgreSQL para Node.js y proporciona todas las herramientas necesarias para establecer conexiones con el gestor de base de datos. En lugar de utilizar una conexión simple, el proyecto emplea la clase Pool, la cual implementa un conjunto de conexiones reutilizables conocido como Connection Pool o

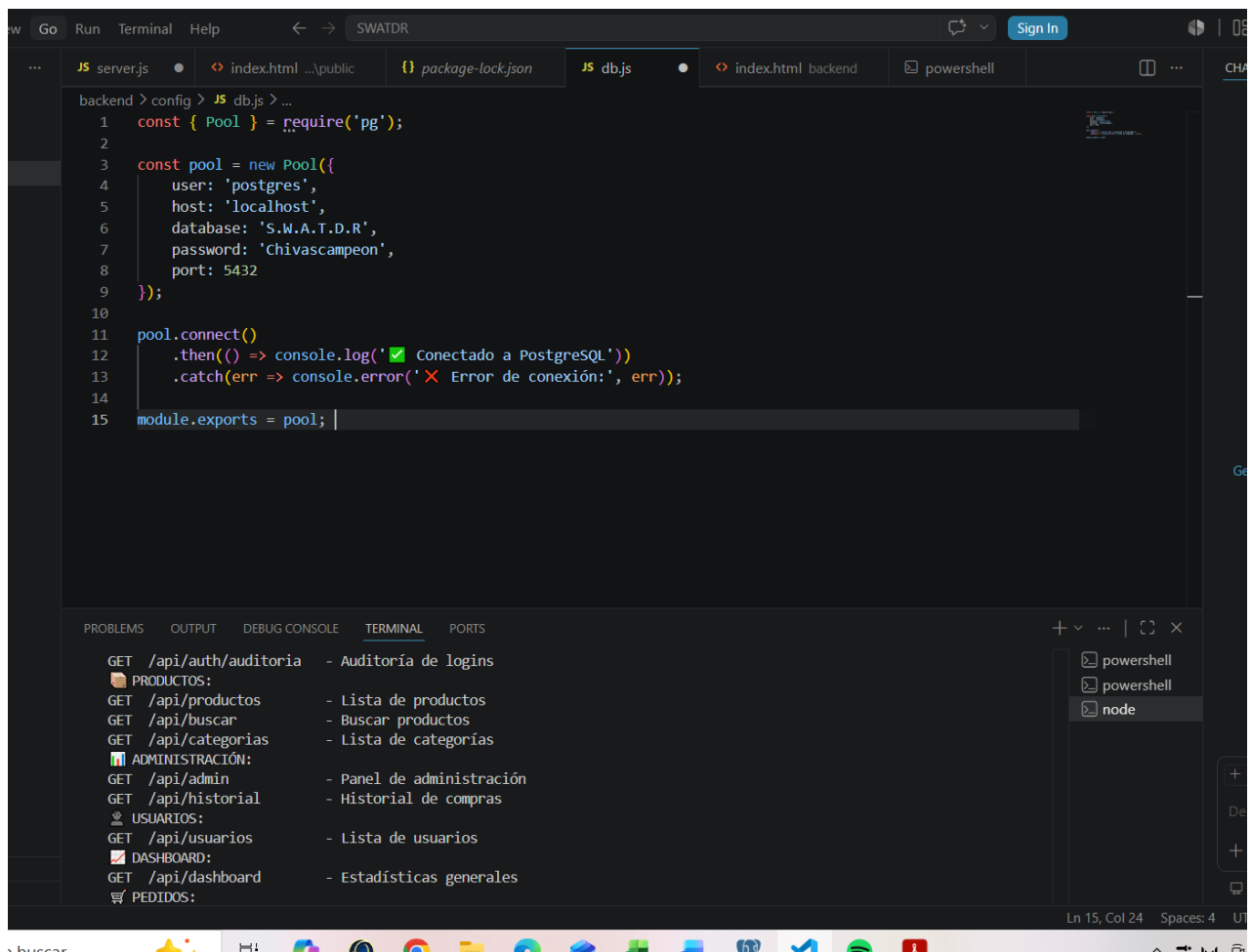
Pool de conexiones. Esta técnica mejora considerablemente el rendimiento del sistema, ya que evita abrir una nueva conexión cada vez que el servidor necesita realizar una consulta. En su lugar, el backend reutiliza conexiones previamente creadas, disminuyendo el tiempo de respuesta y reduciendo el consumo de recursos tanto del servidor como de PostgreSQL.

Posteriormente se crea una instancia del pool mediante la instrucción `const pool = new Pool({...});`. Dentro de este objeto se especifican todos los parámetros necesarios para que Node.js pueda localizar la base de datos. El atributo `user` indica el nombre del usuario con permisos para acceder al servidor PostgreSQL, en este caso `postgres`. El atributo `host` establece la dirección del servidor donde se encuentra alojada la base de datos; al utilizar el valor `localhost` se indica que tanto la aplicación como PostgreSQL se ejecutan en el mismo equipo. El parámetro `database` especifica el nombre exacto de la base de datos que utilizará la aplicación, correspondiente al proyecto S.W.A.T.D.R. Asimismo, el atributo `password` almacena la contraseña asociada al usuario de PostgreSQL y el parámetro `port` define el puerto de comunicación, utilizando el puerto estándar 5432, asignado por defecto al servicio PostgreSQL.

Una vez configurados todos los parámetros de conexión, el archivo realiza una prueba inmediata mediante la función `pool.connect()`. Esta instrucción intenta establecer comunicación con el servidor de base de datos antes de que la aplicación comience a recibir solicitudes de los usuarios. Si la conexión se realiza correctamente, se ejecuta el método `.then()`, mostrando en la consola el mensaje "Conectado a PostgreSQL", indicando que el servidor puede acceder correctamente a la información almacenada. En caso de producirse algún error, como credenciales incorrectas, una base de datos inexistente o un servidor apagado, se ejecuta el método `.catch()`, el cual captura la excepción y muestra un mensaje indicando el motivo del fallo. Este mecanismo facilita la detección de errores durante el desarrollo y evita que la aplicación continúe ejecutándose sin disponer de acceso a la base de datos.

Finalmente, el archivo utiliza la instrucción `module.exports = pool;`, cuya finalidad es exportar el objeto de conexión para que pueda ser utilizado desde cualquier otro módulo del proyecto. Gracias a esta exportación, archivos como `server.js` pueden importar el mismo pool de conexiones mediante la función `require()` y ejecutar consultas SQL sin necesidad de crear nuevas conexiones independientes. Este enfoque favorece la reutilización del código, reduce la duplicidad de configuraciones y mejora la organización del proyecto al centralizar toda la configuración de acceso a PostgreSQL en un único archivo.

En conjunto, el archivo db.js representa el módulo responsable de la comunicación entre el backend y la base de datos. Su implementación mediante un Pool de conexiones permite optimizar el rendimiento del sistema, reducir el consumo de recursos y garantizar que todas las operaciones realizadas por el servidor puedan acceder de manera eficiente y segura a la información almacenada en PostgreSQL. Esta arquitectura constituye una práctica ampliamente utilizada en aplicaciones web modernas debido a su escalabilidad, facilidad de mantenimiento y capacidad para soportar múltiples solicitudes simultáneas.



```
backend > config > JS db.js > ...
1  const { Pool } = require('pg');
2
3  const pool = new Pool({
4    user: 'postgres',
5    host: 'localhost',
6    database: 'S.W.A.T.D.R',
7    password: 'Chivascampeon',
8    port: 5432
9  });
10
11 pool.connect()
12   .then(() => console.log('✅ Conectado a PostgreSQL'))
13   .catch(err => console.error('❌ Error de conexión:', err));
14
15 module.exports = pool;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- GET /api/auth/auditoria - Auditoría de logins
- PRODUCTOS:
- GET /api/productos - Lista de productos
- GET /api/buscar - Buscar productos
- GET /api/categorias - Lista de categorías
- ADMINISTRACIÓN:
- GET /api/admin - Panel de administración
- GET /api/historial - Historial de compras
- USUARIOS:
- GET /api/usuarios - Lista de usuarios
- DASHBOARD:
- GET /api/dashboard - Estadísticas generales
- PEDIDOS:

Ln 15, Col 24 Spaces: 4

(db.js)

-. Frontend (index.html)

El archivo index.html constituye el frontend del sistema SWATDR, es decir, la parte de la aplicación con la que interactúan directamente los usuarios mediante un navegador web. Fue desarrollado utilizando las tecnologías HTML5, CSS3 y JavaScript, las cuales conforman los tres pilares fundamentales del desarrollo web. Este archivo tiene la responsabilidad de presentar la interfaz gráfica del sistema, capturar las acciones realizadas por el usuario y comunicarse con el backend para solicitar o enviar información a la base de datos. En conjunto, el frontend representa la capa de presentación dentro de la arquitectura cliente-servidor implementada en el proyecto.

El desarrollo de este componente se realizó en Visual Studio Code, aprovechando sus herramientas para edición de código, depuración y administración del proyecto. Una característica importante de este archivo es que integra en un mismo documento la estructura de la página, el diseño visual y la lógica del cliente. Aunque en proyectos de gran tamaño estas tecnologías suelen separarse en archivos independientes, para este sistema se decidió concentrarlas en un único documento con el propósito de facilitar su distribución y ejecución durante el desarrollo.

La estructura del documento inicia con la declaración `<!DOCTYPE html>`, indicando al navegador que el archivo utiliza el estándar HTML5. Posteriormente se define el idioma del documento como español mediante el atributo `lang="es"`, lo cual facilita la accesibilidad y el correcto funcionamiento de los motores de búsqueda. Dentro de la sección `<head>` se configuran diferentes elementos necesarios para el funcionamiento de la página, como la codificación de caracteres UTF-8, que permite mostrar correctamente letras acentuadas y caracteres especiales, así como la configuración de la ventana gráfica para que el diseño pueda adaptarse correctamente a diferentes tamaños de pantalla.

En esta misma sección también se establece el título de la página, identificado como "SWATDR - Panel de Control", el cual aparece en la pestaña del navegador y permite identificar la aplicación mientras se encuentra en ejecución.

Después del encabezado comienza la implementación de las hojas de estilo mediante código CSS. Esta parte del archivo tiene como objetivo definir la apariencia visual de todos los elementos que conforman la interfaz. Se establecen características como colores, tipografías, tamaños, márgenes, bordes, sombras, animaciones y distribuciones de los componentes dentro de la página. El diseño utiliza una combinación de colores degradados

para proporcionar una apariencia moderna, además de tarjetas, botones y tablas con esquinas redondeadas que mejoran la experiencia visual del usuario.

Uno de los aspectos más importantes del diseño consiste en la utilización de Flexbox, una técnica de CSS que facilita la distribución automática de los elementos dentro de la pantalla. Gracias a este mecanismo, componentes como el encabezado, los botones y las tarjetas estadísticas pueden reorganizarse automáticamente cuando la aplicación es utilizada desde diferentes dispositivos, logrando un diseño adaptable o Responsive Design. Asimismo, se incorpora una regla @media, la cual modifica el tamaño de diversos elementos cuando la pantalla posee un ancho reducido, permitiendo que la aplicación pueda visualizarse correctamente tanto en computadoras como en dispositivos móviles.

Dentro del cuerpo del documento se construye la interfaz principal del sistema utilizando etiquetas HTML. El elemento central corresponde a un contenedor que agrupa todos los componentes visuales de la aplicación. En la parte superior se encuentra el encabezado del sistema, donde se muestra el nombre SWATDR junto con un panel destinado a visualizar la información del usuario autenticado. Este panel permanece oculto hasta que el usuario inicia sesión correctamente y posteriormente muestra el nombre, el rol asignado y un botón para cerrar la sesión.

Debajo del encabezado se encuentra un conjunto de botones que representan las principales funciones disponibles dentro del sistema. Cada botón se encuentra asociado a un servicio diferente del backend, permitiendo consultar productos, categorías, usuarios, panel administrativo, historial de compras, auditoría de accesos, dashboard estadístico, iniciar sesión, registrar usuarios y limpiar la información mostrada en pantalla. De esta manera, el usuario puede acceder rápidamente a las diferentes funcionalidades sin necesidad de escribir comandos o realizar consultas manuales.

Otro componente importante del frontend corresponde al contenedor identificado como data-container, cuya finalidad es mostrar dinámicamente la información recibida desde el servidor. Inicialmente este espacio presenta un mensaje indicando que el usuario debe seleccionar alguna opción. Sin embargo, una vez que se realiza una consulta al backend, el contenido del contenedor cambia automáticamente para mostrar tablas, mensajes informativos, estadísticas o notificaciones de error según la respuesta obtenida.

El sistema también incorpora dos ventanas modales destinadas a la autenticación. La primera corresponde al formulario de inicio de sesión, donde el usuario introduce su correo electrónico y contraseña. La segunda ventana permite registrar nuevos usuarios mediante

la captura de su nombre, correo electrónico, contraseña y rol correspondiente. Estas ventanas permanecen ocultas hasta que el usuario presiona el botón correspondiente, mejorando la organización visual de la aplicación al evitar cargar formularios innecesarios sobre la pantalla principal.

La lógica del frontend se implementa mediante código JavaScript incluido al final del documento. Este lenguaje es el encargado de proporcionar dinamismo a la página y permitir la comunicación con el backend mediante solicitudes HTTP. Para realizar dichas comunicaciones se utiliza la función `fetch()`, la cual envía peticiones al servidor Node.js y recibe las respuestas en formato JSON. Gracias a este mecanismo, el navegador puede consultar información almacenada en PostgreSQL sin necesidad de recargar completamente la página, proporcionando una experiencia de uso mucho más fluida.

Una de las funciones principales implementadas es `fetchData()`. Su responsabilidad consiste en recibir la dirección de un servicio web, enviar la solicitud correspondiente al backend y esperar la respuesta generada por el servidor. Mientras la consulta se encuentra en proceso, el sistema muestra un mensaje indicando que los datos están siendo cargados. Una vez recibida la respuesta, la información es enviada a la función encargada de construir dinámicamente la tabla correspondiente. En caso de presentarse algún problema, como la ausencia del servidor o un error interno, la aplicación muestra un mensaje descriptivo indicando la causa del fallo.

La función `renderTable()` constituye uno de los componentes más importantes del frontend. Su propósito consiste en analizar automáticamente la estructura de la información recibida y generar tablas HTML sin necesidad de programarlas manualmente para cada consulta. Primero obtiene el nombre de todas las columnas utilizando el método `Object.keys()`. Posteriormente construye el encabezado de la tabla y finalmente recorre todos los registros mediante ciclos repetitivos, insertando la información dentro de las celdas correspondientes. Durante este proceso también realiza diferentes transformaciones, como dar formato monetario a los precios, representar visualmente los estados mediante etiquetas de colores y truncar textos demasiado largos para mejorar la presentación de la información.

Además de mostrar tablas, el sistema genera automáticamente tarjetas estadísticas cuando consulta determinados servicios. Por ejemplo, al acceder al Dashboard se presentan indicadores relacionados con el número total de usuarios, productos, pedidos e ingresos registrados por la tienda. De igual forma, cuando se consulta la auditoría de accesos, el

sistema calcula automáticamente el número de inicios de sesión exitosos y fallidos, proporcionando información resumida sin necesidad de realizar cálculos adicionales en la base de datos.

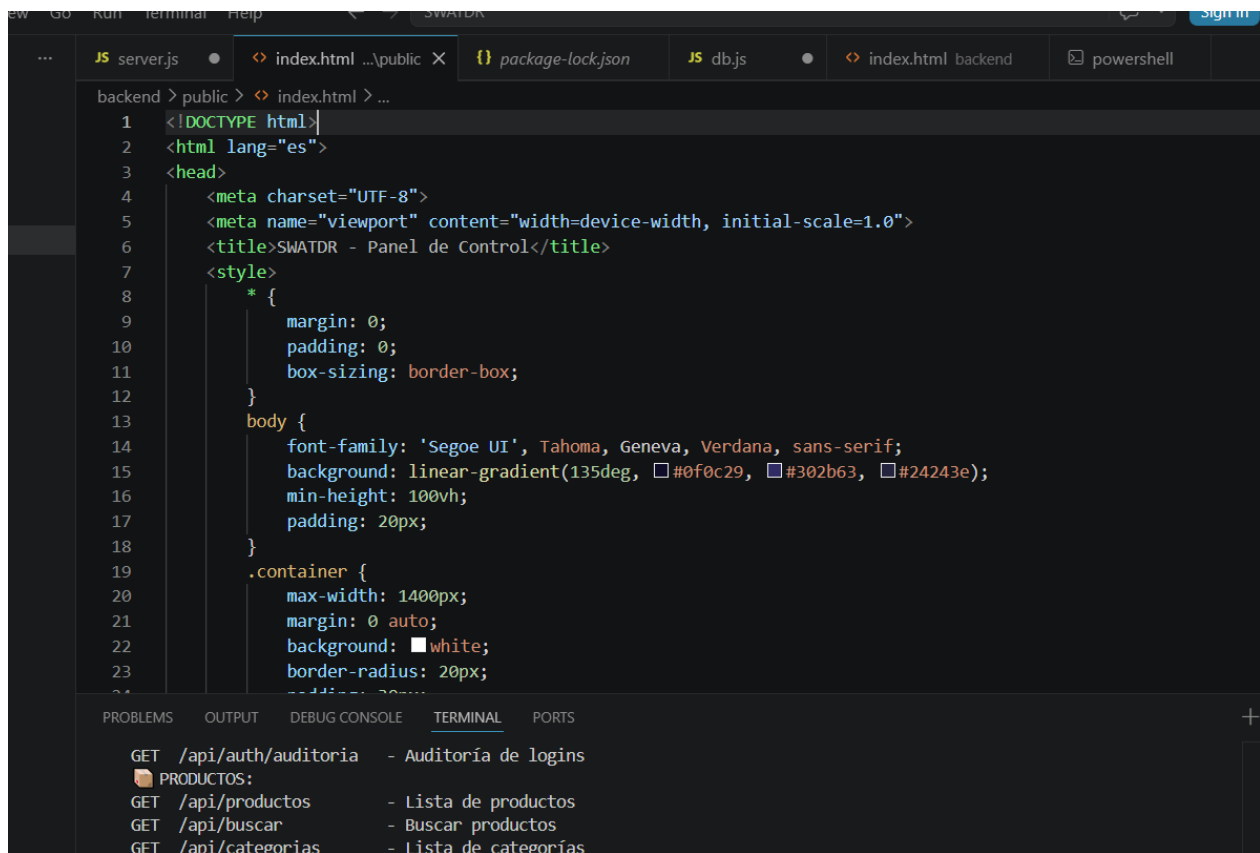
El proceso de autenticación también es administrado completamente desde el frontend. Cuando el usuario intenta iniciar sesión, la función `loginUser()` obtiene las credenciales introducidas en el formulario y las envía al backend mediante una solicitud HTTP POST. El servidor procesa la autenticación utilizando PostgreSQL y devuelve una respuesta indicando si el acceso fue autorizado. Si las credenciales son correctas, el frontend actualiza automáticamente la información del usuario mostrado en pantalla mediante la función `updateUserUI()`, habilitando el panel de usuario y ocultando el formulario de autenticación. En caso contrario, se presenta un mensaje de error indicando el motivo por el cual el acceso fue rechazado.

De manera similar, la función `registerUser()` permite registrar nuevos usuarios dentro del sistema. La información capturada en el formulario es enviada al backend, donde posteriormente será validada y almacenada en la base de datos. Una vez completado el registro exitosamente, el frontend informa al usuario mediante un mensaje de confirmación y actualiza automáticamente el listado de usuarios registrados.

Otra característica importante consiste en la implementación de la función `logout()`, cuya finalidad es cerrar la sesión del usuario actual. Al ejecutarse, elimina la información del usuario autenticado, oculta el panel de información personal y limpia el contenido mostrado en pantalla, devolviendo la aplicación a su estado inicial.

Finalmente, el documento incorpora una función de inicialización mediante `window.onload`. Esta instrucción se ejecuta automáticamente cuando la página termina de cargarse y realiza una consulta inicial al catálogo de productos, permitiendo que el usuario visualice información desde el primer momento sin necesidad de realizar ninguna acción adicional.

En conjunto, el archivo `index.html` constituye la capa de presentación del sistema SWATDR. Mediante la combinación de HTML para la estructura, CSS para el diseño y JavaScript para la lógica del cliente, el frontend proporciona una interfaz gráfica moderna, dinámica y fácil de utilizar. Asimismo, establece comunicación permanente con el backend desarrollado en Node.js mediante una API REST, permitiendo consultar, registrar y visualizar información almacenada en PostgreSQL de forma rápida y eficiente. Esta arquitectura favorece la separación de responsabilidades entre la interfaz, la lógica del servidor y la base de datos, facilitando el mantenimiento, la escalabilidad y la evolución futura del sistema.



```
backend > public > < index.html > ...
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>SWATDR - Panel de Control</title>
7   <style>
8     * {
9       margin: 0;
10      padding: 0;
11      box-sizing: border-box;
12    }
13    body {
14      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
15      background: linear-gradient(135deg, #0f0c29, #302b63, #24243e);
16      min-height: 100vh;
17      padding: 20px;
18    }
19    .container {
20      max-width: 1400px;
21      margin: 0 auto;
22      background: white;
23      border-radius: 20px;
24      padding: 20px;
25    }
26  </style>
27 </head>
28 <body>
29 </body>
30 </html>
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

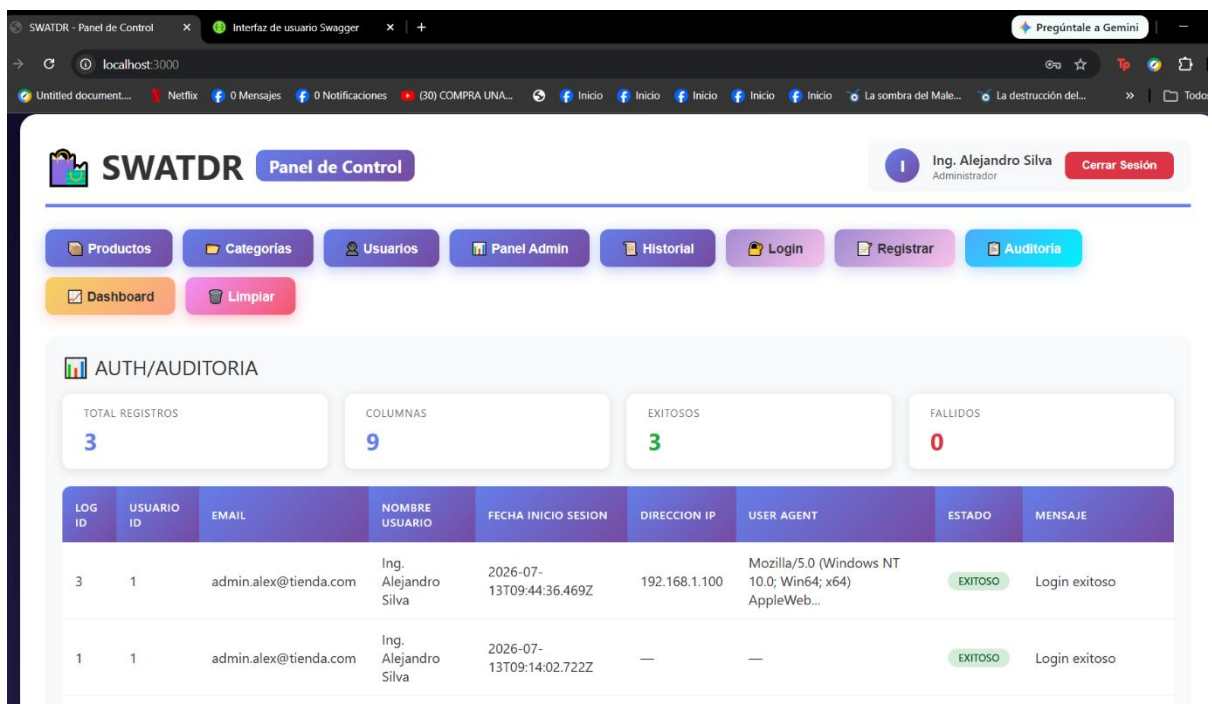
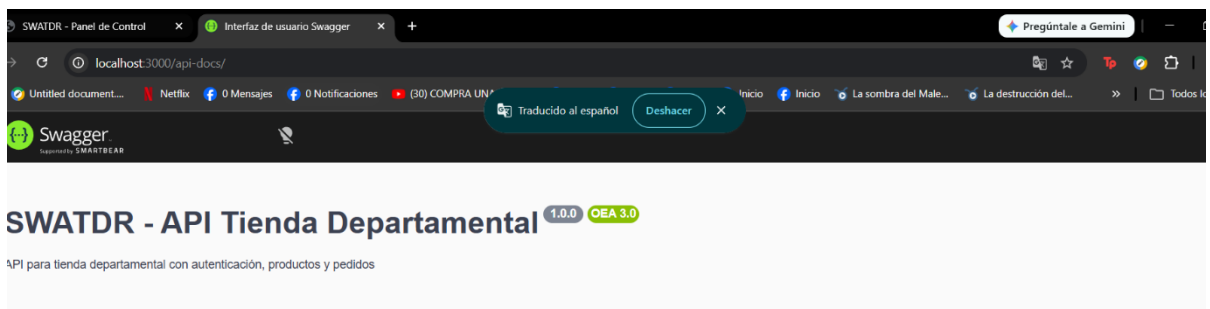
```
GET /api/auth/auditoria - Auditoría de logins
PRODUCTOS:
GET /api/productos - Lista de productos
GET /api/buscar - Buscar productos
GET /api/categorias - Lista de categorías
```

(index.html)

-. Prueba y resultado del sistema

Una vez que se implementaron el frontend, el backend y la base de datos, se llevaron a cabo pruebas de funcionamiento para comprobar que todos los componentes del sistema se integraran correctamente. La primera imagen de la interfaz con la que se gira el sistema es la de Swagger, la cual se usa para documentar y probar la API REST desarrollada en Node.js. Desde esta interfaz se pueden visualizar todos los endpoints disponibles (inicio de sesión, registro de usuarios, consulta de auditorías, consulta de historial de accesos...), así como llamar a cada uno de ellos de forma independiente para comprobar que se obtienen las respuestas correctas, La segunda imagen que tenemos a continuación pertenece a la interfaz principal del sistema SWATDR, donde se puede ver el panel de control una vez un administrador ha entrado al sistema. En esta pantalla están disponibles las distintas funcionalidades del sistema, como: consulta de productos, consulta de categorías, consulta de usuarios, consulta del historial de compras, funcionalidad del panel administrativo, dashboard y funcionalidad de auditoría. En el ejemplo, se ilustra el apartado de Auditoría,

donde se exponen todos los registros de inicio de sesión almacenados mediante el sistema en la base de datos, detalles que incluyen el identificador de los usuarios, el correo electrónico, la fecha y hora de acceso, la dirección IP, el navegador utilizado, el estado del inicio de la sesión y el contenido del mensaje que resulta de cada inicio de sesión. Estos resultados son prueba de que la comunicación entre el frontend, el backend y PostgreSQL opera correctamente, de tal forma que se pueden consultar y administrar de forma ágil y efectiva la información en el sistema.



-.Diccionario de Datos

TABLA: ROLES

Almacena los diferentes perfiles o roles de acceso asignados a los usuarios del sistema.

COLUMNA	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN
rol_id	SERIAL	PRIMARY KEY	Identificador único autoincrementable de cada rol.
nombre_rol	VARCHAR(50)	UNIQUE, NOT NULL	Nombre descriptivo del rol (ej. 'Administrador', 'Comprador').

TABLA: USUARIOS

Registra las credenciales de acceso, información personal y estado de seguridad de las cuentas de usuario.

COLUMNA	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN
usuario_id	SERIAL	PRIMARY KEY	Identificador único autoincrementable del usuario.
nombre	VARCHAR(100)	NOT NULL	Nombre completo del usuario.
email	VARCHAR(150)	UNIQUE, NOT NULL	Correo electrónico corporativo o personal, utilizado como login.
contrasenia_hash	VARCHAR(255)	NOT NULL	Clave de acceso encriptada mediante algoritmo seguro (bcrypt).
rol_id	INT	FK, NOT NULL	Clave foránea que referencia a roles(rol_id). Restricción ON DELETE RESTRICT.
fecha_registro	TIMESTAMP	DEFAULT NOW()	Fecha y hora exacta del registro del usuario.
intentos_fallidos	INT	DEFAULT 0	Contador interno de intentos incorrectos de login para control de bloqueos.
bloqueado_hasta	TIMESTAMP	DEFAULT NULL	Fecha y hora límite de penalización en caso de múltiples intentos erróneos.

TABLA: CATEGORIAS

Agrupa los productos bajo clasificaciones comerciales específicas.

COLUMNA	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN
categoria_id	SERIAL	PRIMARY KEY	Identificador único de la categoría.
nombre_categoria	VARCHAR(100)	UNIQUE, NOT NULL	Nombre único de la categoría (ej. 'Caballeros', 'Damas').
descripcion	TEXT	-	Detalle o descripción ampliada del tipo de productos de esta categoría.

TABLA: PRODUCTOS

Catálogo maestro de artículos disponibles en la tienda departamental.

COLUMNA	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN
producto_id	SERIAL	PRIMARY KEY	Identificador único comercial del producto.
nombre_producto	VARCHAR(150)	NOT NULL	Nombre comercial del artículo.
descripcion	TEXT	-	Ficha técnica o descripción comercial de la prenda o artículo.
categoria_id	INT	FK, NOT NULL	Clave foránea vinculada a categorias(categoria_id). ON DELETE RESTRICT.
precio_mercado	NUMERIC(10,2)	NOT NULL, CHECK	Precio base de venta. Debe ser mayor estricto a cero (> 0).
marca	VARCHAR(100)	-	Fabricante o marca comercial del producto.
fecha_ingreso	TIMESTAMP	DEFAULT NOW()	Fecha en la que el producto fue ingresado al sistema.

TABLA: VARIANTES_PRODUCTO

Desglose dimensional de los productos según combinaciones específicas de talla y color.

COLUMNA	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN
variante_id	SERIAL	PRIMARY KEY	Identificador único de la variante del producto.
producto_id	INT	FK, NOT NULL	Clave foránea vinculada a productos(producto_id). ON DELETE CASCADE.
talla	VARCHAR(10)	NOT NULL	Medida o talla física de la prenda (ej. 'CH', 'M', 'G', '28').
color	VARCHAR(50)	NOT NULL	Descripción del color de la variante.
stock	INT	NOT NULL, CHECK	Cantidad física disponible en almacén. Debe ser mayor o igual a cero (≥ 0).

TABLA: PEDIDOS

Cabecera de las transacciones de compra realizadas por los clientes.

COLUMNA	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN
pedido_id	SERIAL	PRIMARY KEY	Número de orden o pedido correlativo.
comprador_id	INT	FK, NOT NULL	Clave foránea hacia usuarios(usuario_id). Representa al cliente comprador.
fecha_pedido	TIMESTAMP	DEFAULT NOW()	Fecha y hora del registro de la compra.
total	NUMERIC(10,2)	DEFAULT 0.00	Monto acumulado total del pedido.

TABLA: DETALLES_PEDIDO

Líneas de renglones específicos asociados a cada pedido (variantes y unidades compradas).

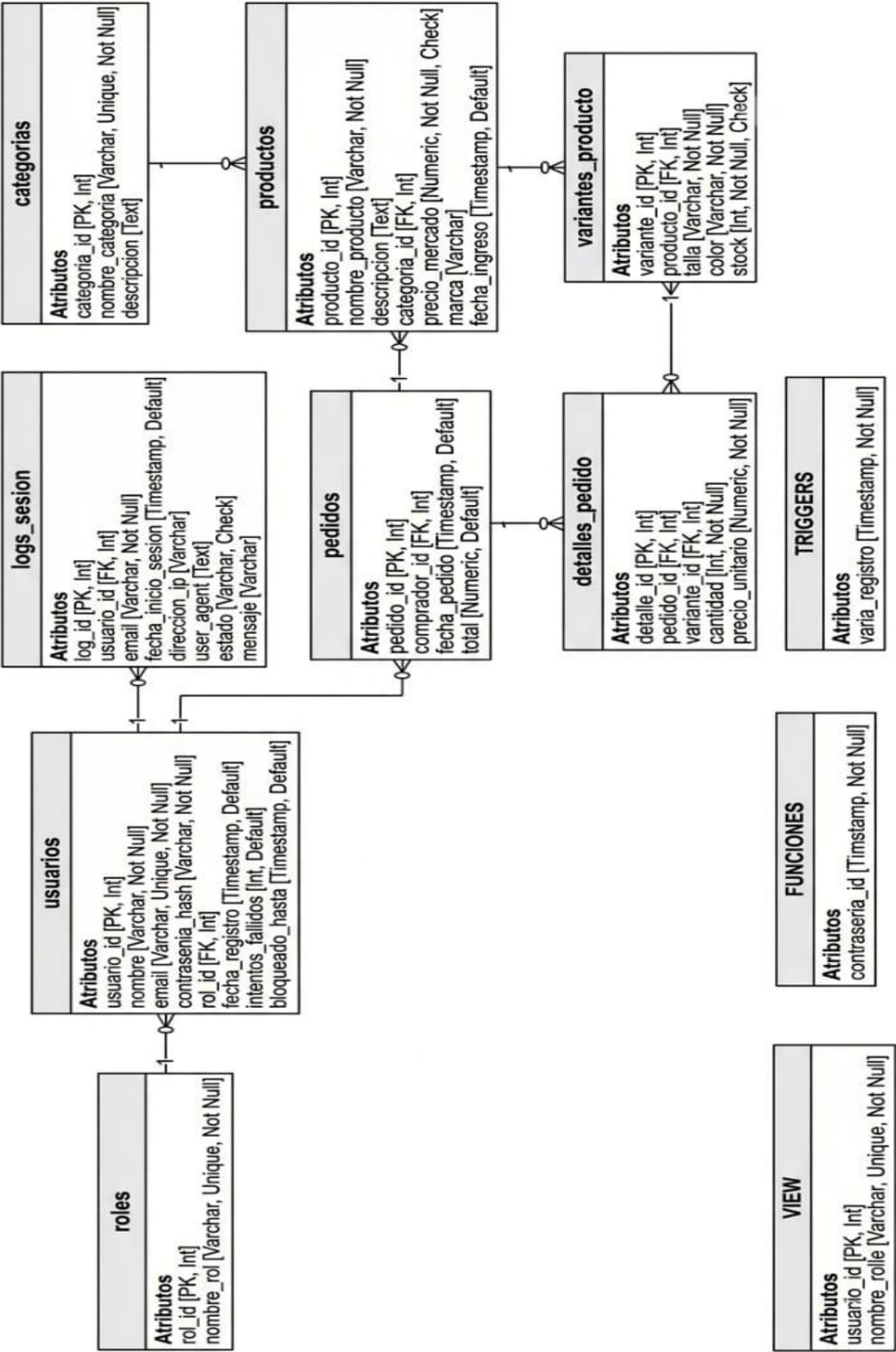
COLUMNA	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN
detalle_id	SERIAL	PRIMARY KEY	Identificador del renglón de la orden.
pedido_id	INT	FK, NOT NULL	Clave foránea vinculada a pedidos(pedido_id). ON DELETE CASCADE.
variante_id	INT	FK, NOT NULL	Clave foránea vinculada a variantes_producto(variante_id). ON DELETE RESTRICT.
cantidad	INT	NOT NULL, CHECK	Unidades adquiridas del producto. Debe ser mayor a cero (> 0).
precio_unitario	NUMERIC(10,2)	NOT NULL	Precio al que se vendió la unidad en el momento de la compra.

TABLA: LOGS_SESION (TABLA DE AUDITORÍA)

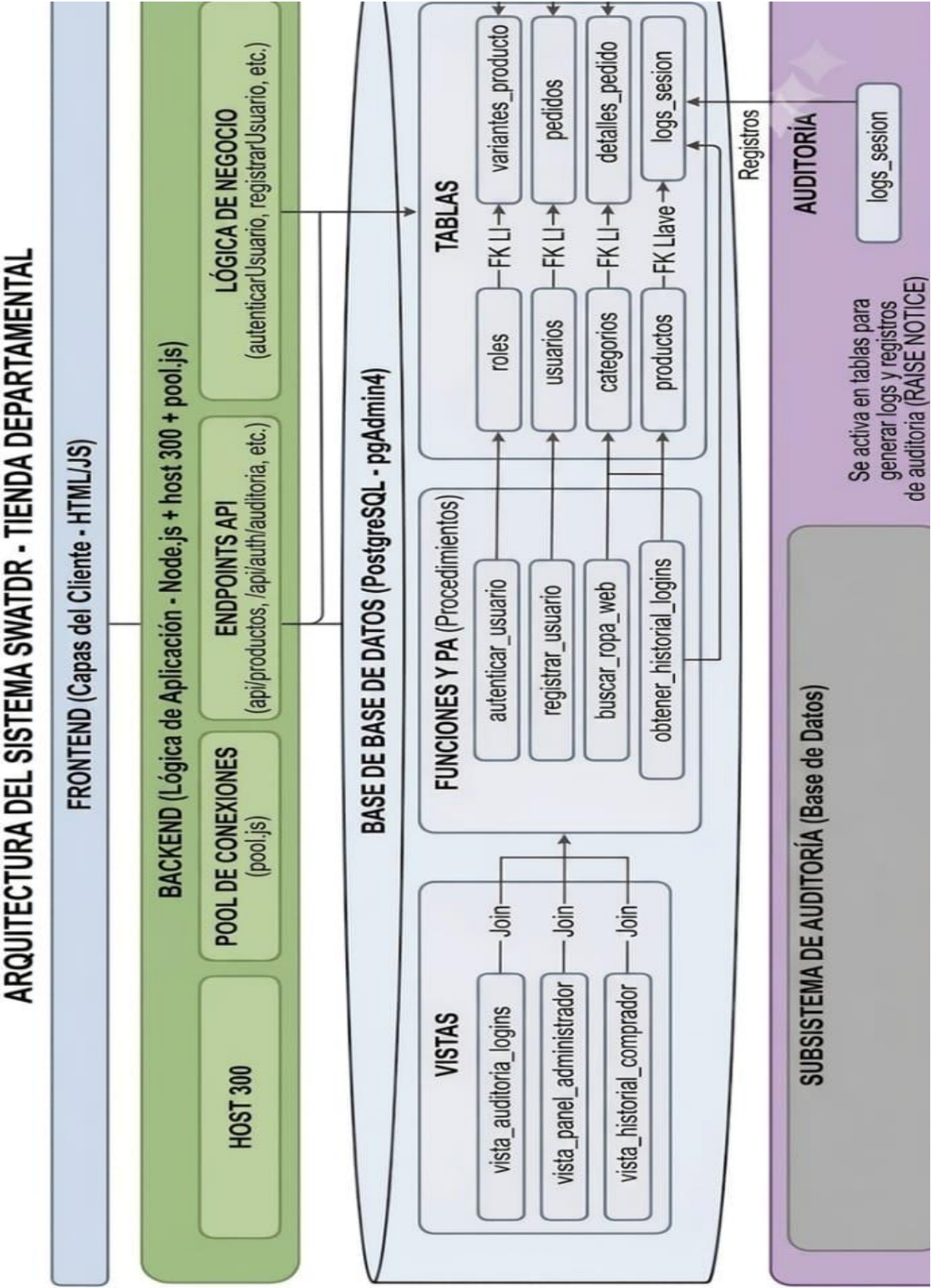
Bitácora centralizada para el monitoreo de intentos de inicio de sesión y control de accesos.

COLUMNA	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN
log_id	SERIAL	PRIMARY KEY	Identificador único del registro de auditoría.
usuario_id	INT	FK, NOT NULL	ID del usuario involucrado. En caso de fallar por inexistencia, toma valor 0.
email	VARCHAR(150)	NOT NULL	Correo electrónico con el que se intentó iniciar sesión.
fecha_inicio_sesion	TIMESTAMP	DEFAULT NOW()	Marca temporal exacta del evento de login.
direccion_ip	VARCHAR(45)	-	Dirección IP de origen del cliente (soporta IPv4 e IPv6).
user_agent	TEXT	-	Cadena de agente de usuario del navegador o cliente web.
estado	VARCHAR(20)	NOT NULL, CHECK	Resultado del intento de acceso. Valores válidos: 'EXITOSO', 'FALLIDO'.
mensaje	VARCHAR(255)	-	Detalle de la auditoría (ej. 'Contraseña incorrecta', 'Login exitoso').

Diagrama Entidad Relación

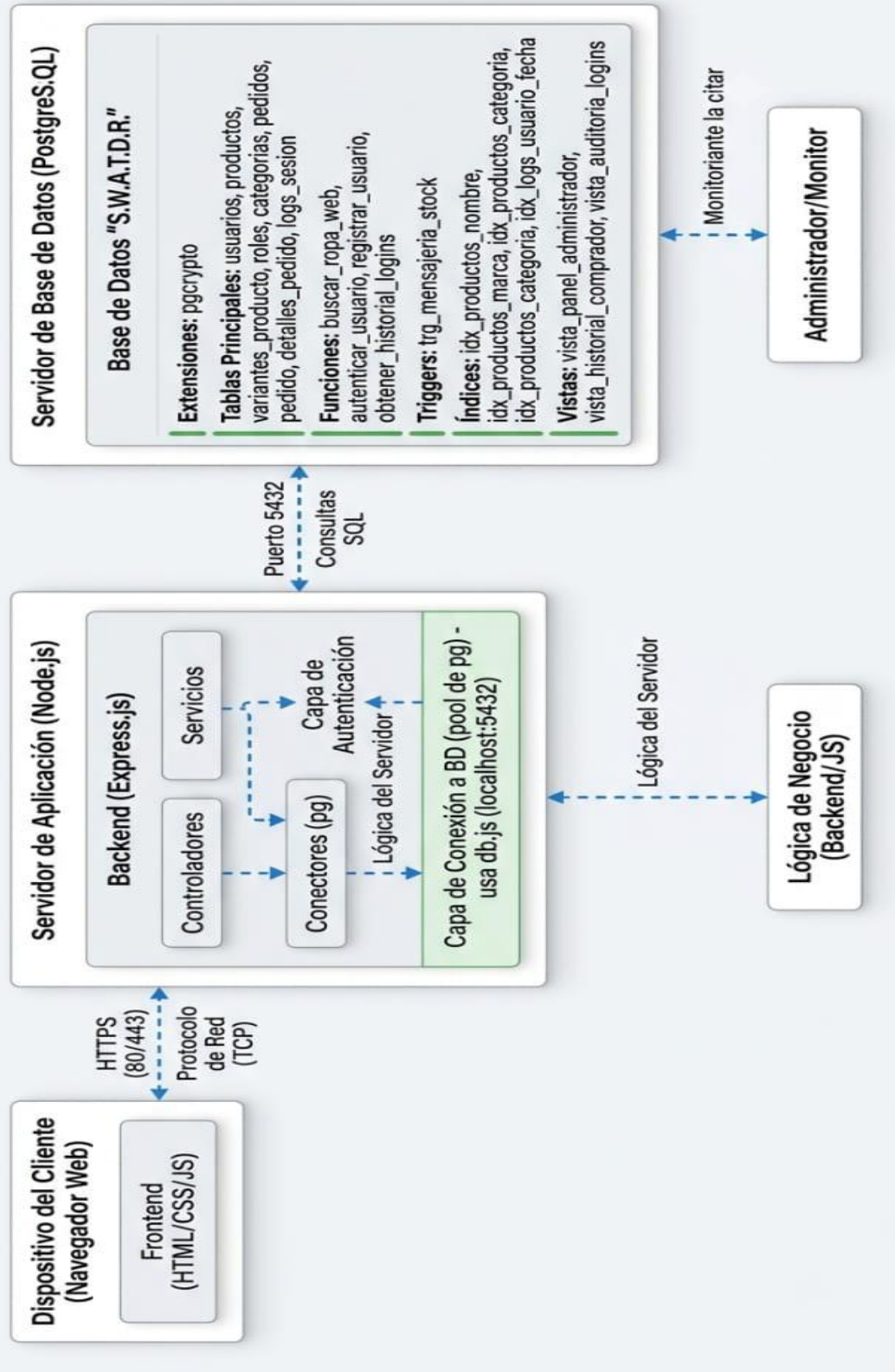


- Diagrama de arquitectura



-. Diagrama de despliegue

Diagrama de Despliegue de la Aplicación SWATDR (Tienda Departamental)



-. Explicación del código

El sistema está compuesto por tres capas esenciales: la Base de Datos (SQL) en PostgreSQL con lógica de seguridad embebida, el Servidor de Conexión (Node.js) que actúa como puente, y la Interfaz Gráfica (HTML/CSS) para el control del administrador.

1. Base de Datos (database.sql)

Este script diseña una arquitectura relacional sólida para una tienda departamental de ropa con un enfoque crítico en la seguridad y la automatización:

- Seguridad y Cifrado (pgcrypto): Utiliza la extensión pgcrypto para cifrar las contraseñas en la base de datos usando Blowfish (bcrypt) a través de la función crypt(). Las contraseñas reales nunca se guardan en texto plano.
- Modelo de Datos Completo: Organiza la información mediante tablas de roles, usuarios, categorías, productos y variantes_producto (gestiona combinaciones únicas de talla, color y existencias). Las compras se dividen en un maestro (pedidos) y su detalle (detalles_pedido).
- Políticas de Bloqueo Automático: La función autenticar_usuario implementa una regla estricta: si un usuario se equivoca 5 veces seguidas, la cuenta se bloquea automáticamente por 30 minutos actualizando la columna bloqueado_hasta.
- Disparadores (Triggers) en Tiempo Real: Cuenta con el trigger trg_mensajeria_stock que se ejecuta después de insertar o modificar el inventario. Lanza alertas críticas en consola (RAISE NOTICE) cuando una variante se queda sin existencias (stock = 0) o tiene un inventario bajo (stock < 10).
- Vistas de Auditoría: vista_auditoria_logins recopila las direcciones IP y el navegador (user_agent) desde donde se intentó iniciar sesión, permitiendo rastrear hackeos o accesos fallidos.

2. Conector del Servidor (db.js)

Es el puente de comunicación backend desarrollado en Node.js utilizando el controlador oficial pg (PostgreSQL client):

- Pool de Conexiones: Utiliza un sistema de *Pool* (piscina de conexiones) en lugar de una conexión única. Esto es ideal para producción, ya que permite reutilizar múltiples canales abiertos con PostgreSQL de forma simultánea, ahorrando memoria y procesador ante múltiples peticiones.
- Credenciales del Sistema: Define los parámetros de red local (localhost), puerto por defecto (5432), la base de datos (S.W.A.T.D.R), el usuario administrador y la contraseña del servidor.
-

3. Interfaz de Usuario (index.html)

Es el panel de control frontal (*Dashboard*) diseñado desde cero con estándares modernos:

- Diseño Responsivo y Limpio: Utiliza estilos CSS basados en Flexbox para adaptarse a pantallas de computadoras y tabletas. Utiliza un gradiente oscuro de fondo para dar un aspecto moderno de "Modo Oscuro" o aplicación empresarial de alta gama.
- Identidad de Usuario: Incluye un bloque visualizado en la cabecera que simula el perfil del usuario activo (Ing. Alejandro Silva con rol de Administrador), junto con un avatar generado puramente por estilos CSS.
- Panel Escanearle: La botonera principal (btn-group) usa clases de color contextuales (Éxito en verde, Alertas en amarillo, Peligro en rojo) preparadas para conectarse mediante JavaScript (Fetch API) a las funciones y vistas creadas en el archivo SQL.